

COA — Quiz 3 Study Guide

Pipelined Processor • Memory Hierarchy • Cache Memory

MCQ Format | 10 Questions | 10 Marks

SECTION 1: Pipelined Processor

1.1 What is Pipelining?

- Pipelining improves the throughput of a digital system.
- A single-cycle processor is subdivided into 5 pipeline stages.
- Five instructions can execute simultaneously — one in each stage.
- Each stage has only 1/5th of the logic → clock frequency is ~5× faster.
- Latency per instruction is ideally unchanged; throughput is ideally 5× better.

1.2 The Five Pipeline Stages

Stage	Name	What Happens
1	Fetch (F)	Processor reads the instruction from instruction memory (IM)
2	Decode (D)	Reads source operands from register file (RF); decodes instruction → control signals
3	Execute (E)	ALU performs the computation
4	Memory (M)	Processor reads or writes data memory (DM)
5	Writeback (W)	Result written back to register file (RF)

1.3 Timing: Single-Cycle vs Pipelined

Single-Cycle Processor

- Instruction latency: $200 + 100 + 120 + 200 + 60 = 680$ ps
- Throughput: 1 instruction / 680 ps $\approx$ 1.47 billion instructions/sec
- Second instruction begins only after first completes.

Pipelined Processor

- Pipeline stage length = 200 ps (set by slowest stage — memory access).
- Instruction latency: $5 \times 200 = 1000$ ps (higher than single-cycle!)
- Throughput: 1 instruction / 200 ps = 5 billion instructions/sec

△ Latency INCREASES in pipelined design because stages are not perfectly balanced. Throughput increases but is not exactly 5× due to stage imbalance.

1.4 Pipelined Datapath

- Formed by chopping the single-cycle datapath into 5 stages separated by pipeline registers.
- 4 pipeline registers are inserted between the 5 stages.
- Signals carry a suffix (F, D, E, M, W) indicating which stage they belong to.
- Register file: read in Decode stage, written in Writeback stage.
- Register file writes on the falling edge of CLK → write in first half, read in second half of same cycle (no hazard).
- WA3 (write address) is pipelined through E, M, W stages to stay in sync with the instruction.

1.5 Pipeline Hazards — Overview

Hazard Type	Definition	Solution
Data Hazard (RAW)	Instruction reads a register not yet written by a previous instruction	Forwarding (bypassing) or Stalls
Control Hazard	Branch target unknown when next instruction is fetched	Stall pipeline or flush instructions

1.6 Data Hazard — RAW (Read After Write)

- RAW Hazard: occurs when an instruction needs a result that hasn't been written back yet.
- Example: ADD writes R1 in cycle 5 (Writeback). AND reads R1 in cycle 3 — too early!
- SUB reads R1 in the second half of cycle 5 → gets correct value (no hazard).
- AND and ORR read R1 before it is written → wrong values!

Solution 1: NOP Insertion (Software)

- Insert NOP instructions between ADD and AND to create delay.
- Downside: complicates programming and degrades performance.

Solution 2: Forwarding (Bypassing)

- Forward result from Memory or Writeback stage to the Execute stage of a dependent instruction.
- Cycle 4: R1 forwarded from Memory stage of ADD → Execute stage of AND.
- Cycle 5: R1 forwarded from Writeback stage of ADD → Execute stage of ORR.
- Hardware: Hazard Unit + 2 forwarding multiplexers added in front of ALU.

Forwarding conditions (Hazard Unit logic):

ForwardAE = 10 if Match_1E_M · RegWriteM → SrcAE = ALUOutM

ForwardAE = 01 if Match_1E_W · RegWriteW → SrcAE = ResultW

ForwardAE = 00 otherwise → SrcAE from Register File

Where: Match_1E_M = (RA1E == WA3M), Match_1E_W = (RA1E == WA3W)

Solution 3: Stalls (for LDR hazard)

- LDR finishes reading data only at the END of the Memory stage → cannot forward to the immediately next instruction's Execute stage.
- LDR has a two-cycle latency: a dependent instruction must wait 2 cycles.
- Stall: pipeline is held up, AND stays in Decode stage for one extra cycle.
- Unused stages propagate as bubbles (behave like NOP instructions).
- Stall condition: LDR in Execute AND (RA1D == WA3E OR RA2D == WA3E).
- Stalling a stage: disable its pipeline register (contents don't change).
- When a stage stalls → ALL previous stages also stall; pipeline register after stalled stage is flushed.

1.7 Control Hazard & Branch

- Branch (B instruction): processor doesn't know the next instruction to fetch because branch decision isn't made yet.
- Naïve stall-till-Writeback approach: stall for 4 cycles at every branch.
- Optimization: make branch decision in Execute stage (when destination address and CondEx are known).
- Branch misprediction penalty reduced to only 2 instructions (instead of 4) flushed.
- Hardware changes: branch multiplexer before PC register; BranchTakenE signal from Cond Unit.

SECTION 2: Memory Introduction & Hierarchy

2.1 Principle of Locality

- Programs access a relatively small portion of their address space at any instant.
- Temporal locality: if a location is referenced, it will tend to be referenced again soon. (e.g., loops)
- Spatial locality: if a location is referenced, nearby addresses will also be referenced soon. (e.g., arrays)

2.2 Memory Hierarchy Structure

Level	Technology	Speed	Size	Cost/bit
Closest to CPU	SRAM	Fastest	Smallest	Highest
Middle	DRAM	Medium	Medium	Medium
Farthest from CPU	Magnetic Disk / Flash	Slowest	Biggest	Lowest

2.3 Memory Technologies

SRAM (Static RAM)

- Data stored in transistors; requires constant power.
- Used in caches (close to processor).
- Access time: 0.5–2.5 ns | Cost: \$500–\$1000/GiB

- No refresh needed.

DRAM (Dynamic RAM)

- Data stored as charge in capacitors.
- Used for main memory.
- Requires periodic refresh.
- Access time: 50–70 ns | Cost: \$10–\$20/GiB

Flash Memory

- Electrically Erasable Programmable ROM (EEPROM).
- Non-volatile; used in mobile/portable devices.
- Access time: 5,000–50,000 ns | Cost: \$0.75–\$1.00/GiB

Magnetic Disk

- Largest and slowest memory in servers.
- Platters with magnetic recording material; organized as tracks and sectors.
- Access time: 5,000,000–20,000,000 ns | Cost: \$0.05–\$0.10/GiB

2.4 Key Definitions

Term	Definition
Block / Line	Minimum unit of data that is either present or absent in a level
Hit	Data requested by processor found in the upper (faster) level
Miss	Data NOT found in upper level; must fetch from lower level
Hit Rate	Fraction of memory accesses found in a level
Miss Rate	Fraction of accesses NOT found in a level ($= 1 - \text{Hit Rate}$)
Hit Time	Time to access a level including determining hit or miss
Miss Penalty	Time to replace a block from lower level + deliver it to processor

- Data is copied between only two adjacent levels at a time.
- Goal: give user the capacity of the cheapest technology at the speed of the fastest.

SECTION 3: Cache Memory

3.1 What is Cache?

- Cache = fastest level of memory hierarchy, sits between processor and main memory.
- The memories in the datapath are replaced by caches.
- Word transfer between CPU and cache (fast); block transfer between cache and main memory (slow).

3.2 Cache Mapping Techniques

Mapping	How it works	Key Feature
Direct Mapping	Each main memory block maps to exactly one cache frame	Simple; prone to conflicts
Associative (Fully)	A block can go into ANY cache frame; uses CAM for tag lookup	Flexible; expensive hardware
Set-Associative	Cache divided into sets; a block maps to a specific set but any frame within it	Balance between above two

Direct Mapping — Address Structure

Main memory address is split into: [Tag | Cache Index | Block Offset]

- Cache frame number = (Block address) mod (Number of cache blocks)
- Block address = $\lfloor \text{Byte address} / \text{Bytes per block} \rfloor$

3.3 Worked Numerical Examples

Example 1 — Direct Mapping Block Number

Q: Cache has 64 blocks, block size = 16 bytes. Which cache block does byte address 1200 map to?

Step 1: Block address = $\lfloor 1200 / 16 \rfloor = 75$

Step 2: Cache block = $75 \bmod 64 = 11$

Answer: Cache block 11 (covers byte addresses 1200–1215)

Example 2 — Set-Associative Mapping (Doubling Associativity)

Q: A processor cache has 128 frames. Associativity is doubled (capacity & block size unchanged). What is guaranteed NOT to change?

Given: 16-bit memory address, 4-bit offset $\rightarrow$ 12-bit page address [Tag | Set | Offset]

2-way SA: Sets = $128/2 = 64 = 2^6 \rightarrow$ Address = [t bits | 6-bit Set | 4-bit offset]

4-way SA: Sets = $128/4 = 32 = 2^5 \rightarrow$ Address = [t bits | 5-bit Set | 4-bit offset]

When associativity doubles: Set index bits decrease by 1, Tag bits increase by 1.

Answer: D — Width of processor-to-main-memory data bus is NOT affected.

(Tag comparator width increases; Set Index Decoder width decreases; Way selector MUX width increases)

Example 3 — Cache Size Formula

Q: A direct-mapped cache with 1K blocks, each block = 32 bytes. What is the block offset field width?

Block offset bits = $\log_2(32) = 5$ bits

Index bits = $\log_2(1024) = 10$ bits

Example 4 — Miss Rate Interpretation

Q: If hit rate = 0.95, miss rate = 0.05. If hit time = 2 ns and miss penalty = 100 ns, what is average memory access time?

AMAT = Hit time + Miss rate $\times$ Miss penalty

AMAT = 2 + 0.05 $\times$ 100 = 2 + 5 = 7 ns

3.4 Handling Cache Misses

Instruction Cache Miss

Steps taken on a miss:

1. Send original PC (current PC – 4) to memory.
2. Stall processor; wait for main memory to complete the read.
3. Write cache entry: data in data field, upper address bits in tag, set valid bit ON.
4. Restart instruction execution — re-fetch; now found in cache.

3.5 Write Policies

Policy	How it works	Issue
Write-Through	Every write updates BOTH cache AND main memory immediately	Slow; every write hits main memory
Write Buffer	Write to cache + write buffer; processor continues; buffer drains to memory	Stall if write buffer is full
Write-Back	Write only to cache block; write to main memory ONLY when block is replaced	Inconsistency until replaced

⚠ Write-through ensures consistency but is slow. Write-back is faster but cache and memory may be inconsistent temporarily.

SECTION 4: Quick Revision — MCQ Tips

Likely Question Area	Key Fact to Remember
Pipeline throughput	Pipelined: 1 instr / 200 ps = 5 GHz Single-cycle: 1/680 ps $\approx$ 1.47 GHz
Pipeline latency	Pipelined latency (1000 ps) > Single-cycle (680 ps) — stages not balanced
RAW hazard	Instruction reads register before prior instruction writes it back
Forwarding path	Memory→Execute and Writeback→Execute; handled by Hazard Unit
LDR stall condition	LDR has 2-cycle latency; next instruction MUST stall (forwarding cannot help)
Bubble / NOP	Unused stage propagating through pipeline due to stall
Branch penalty (optimized)	2 instructions flushed when branch decided in Execute stage

Temporal locality	Loops — recently used data reused soon
Spatial locality	Arrays — nearby addresses accessed soon
SRAM vs DRAM	SRAM: transistors, fast, no refresh DRAM: capacitors, slower, needs refresh
Cache block address	$\lfloor \text{Byte address} / \text{Block size} \rfloor$
Direct mapping formula	Cache block = Block address mod Number of cache blocks
AMAT formula	$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$
Write-through vs Write-back	Write-through: always consistent Write-back: faster but temporarily inconsistent
Valid bit in cache	Tells whether a cache entry holds valid data (set ON after first load)
WA3 pipelining	Write address pipelined through $E \rightarrow M \rightarrow W$ to stay in sync with instruction

Good luck on your quiz!